

Secure Coding Review Report

CodeAlpha Cyber Security Internship

Submitted By

Name: Janith Sai

Domain: Cyber Security

Task: Task 3 – Secure Coding Review

1. Introduction

Secure coding is an essential practice in cybersecurity that helps developers create applications resistant to attacks and vulnerabilities. Weak coding practices can expose applications to risks such as unauthorized access, data leakage, and password theft.

This project focuses on reviewing a Python-based login application to identify security vulnerabilities and implement secure coding practices.

2. Objective

The main objectives of this project are:

- To analyze insecure coding practices
 - To identify vulnerabilities in a login application
 - To understand risks associated with weak authentication systems
 - To implement secure coding improvements
 - To learn secure software development practices
-

3. Tools and Technologies Used

Tool / Technology Purpose

Python	Programming Language
VS Code	Code Editor
hashlib	Password Hashing
Windows OS	Operating System

4. Project Description

The project reviews a simple Python login system that initially contains security weaknesses such as hardcoded credentials and plain text password handling.

The vulnerable version was analyzed to identify security issues. A secure version of the application was then developed using password hashing techniques to improve authentication security.

5. Vulnerable Application Code

```
username = input("Enter Username: ")  
password = input("Enter Password: ")
```

```
if username == "admin" and password == "admin123":  
    print("Login Successful")  
else:  
    print("Invalid Credentials")
```

6. Vulnerabilities Identified

6.1 Hardcoded Credentials

The username and password are directly written inside the source code.

Risk

If attackers gain access to the source code, they can easily view the credentials.

Recommendation

Store credentials securely using databases or environment variables.

6.2 Plain Text Passwords

Passwords are compared in plain text format.

Risk

Plain text passwords are insecure and vulnerable to theft.

Recommendation

Use password hashing algorithms such as SHA-256 or bcrypt.

6.3 Lack of Input Validation

The application does not validate or sanitize user inputs.

Risk

Improper input handling may lead to injection attacks or unexpected behavior.

Recommendation

Validate and sanitize all user inputs.

7. Secure Version of the Application

```
import hashlib
```

```
stored_username = "admin"
```

```
stored_password = hashlib.sha256("admin123".encode()).hexdigest()
```

```
username = input("Enter Username: ")
```

```
password = input("Enter Password: ")
```

```
hashed_password = hashlib.sha256(password.encode()).hexdigest()
```

```
if username == stored_username and hashed_password == stored_password:
```

```
    print("Login Successful")
```

```
else:
```

```
    print("Invalid Credentials")
```

8. Explanation of Improvements

The secure version improves application security by:

- Using password hashing
 - Avoiding direct password comparison
 - Demonstrating secure authentication practices
 - Reducing the risk of password exposure
-

9. Output

Successful Login

Enter Username: admin

Enter Password: admin123

Login Successful

Invalid Login

Enter Username: user

Enter Password: wrongpass

Invalid Credentials

10. Secure Coding Best Practices

- Avoid hardcoded credentials
 - Use encrypted or hashed passwords
 - Validate user inputs
 - Follow authentication best practices
 - Use least privilege principles
 - Keep software updated
-

11. Advantages of Secure Coding

- Improves application security
 - Reduces risk of cyber attacks
 - Protects user data
 - Prevents unauthorized access
 - Enhances software reliability
-

12. Conclusion

The Secure Coding Review project successfully identified vulnerabilities in a Python login application and implemented security improvements using password hashing techniques. The project provided practical understanding of secure authentication methods and secure software development practices.

13. Future Improvements

Future enhancements may include:

- Database integration
- Multi-factor authentication
- Advanced encryption methods

- Secure session management
 - Role-based access control
-

14. References

- Python Official Documentation
- OWASP Secure Coding Practices
- CodeAlpha Internship Guidelines